



Rapport de Projet : MLP et moteur d'autodiff appliqués à MNIST

Auteurs:

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

Encadrant:

Aurélien Delval

Responsables:

Aurélien Delval / Hugo Bolloré / Pablo Oliveira

December 19, 2025

Sommaire

1 Introduction

Ce document présente le rapport du projet de construction d'un réseau de neurones multicouche (MLP) et d'un moteur de différenciation automatique appliqué au jeu de données MNIST [?]. Réalisé dans le cadre de l'UE Projet en première année de master de calcul haute performance, simulation à l'Université Paris Saclay, ce travail a pour objectif d'implémenter pas à pas les fondements d'un système d'apprentissage automatique sans dépendre de bibliothèques haut niveau.

L'objectif général du projet est double :

1. Comprendre et implémenter les mécanismes internes d'un réseau neuronal, incluant le passage avant, la rétropropagation et la mise à jour de paramètres.
2. Construire un moteur d'autodiff permettant de dériver automatiquement les expressions mathématiques du réseau, afin de calculer les gradients nécessaires à l'apprentissage.

En parallèle, le projet vise à familiariser les étudiants avec l'analyse expérimentale, l'optimisation du code et la préparation aux travaux de performance qui seront approfondis au second semestre.

2 Guide de lecture

Le rapport est structuré de manière à accompagner progressivement le lecteur :

- La première partie présente le contexte, le problème à résoudre et un bref état de l'art autour de MNIST et des MLP.
- La deuxième partie décrit en détail l'implémentation : choix de conception, organisation du code et intégration des différentes phases.
- La troisième partie expose les expérimentations réalisées sur MNIST ainsi que les observations associées.
- La quatrième partie propose une première analyse de performances (profiling) afin d'identifier les points critiques qui seront optimisés au second semestre.
- Un glossaire et des références sont placés en fin de document pour clarifier les notions techniques.

3 Contexte et État de l'art

3.1 Introduction au problème

Au cours des dernières décennies, l'apprentissage automatique a connu un développement considérable, principalement grâce aux progrès réalisés dans le domaine des réseaux de neurones artificiels. Parmi les problèmes emblématiques de ce domaine figure la reconnaissance de chiffres manuscrits, couramment étudiée à l'aide du jeu de données MNIST [?], devenu un standard pour les tâches de classification. Ce travail vise à présenter les approches classiques permettant de résoudre ce problème, en commençant par le perceptron multicouche (MLP), avant d'aborder ses principales variantes et l'évolution des modèles plus récents.

3.2 Concept de la méthode classique

Le problème MNIST consiste à classer des images de chiffres manuscrits (28×28 pixels) dans dix catégories correspondant aux chiffres 0 à 9. Pour résoudre ce problème, on utilise un perceptron

multicouche (MLP), c'est-à-dire un réseau de neurones constitué de plusieurs neurones artificiels organisés en couches (entrée, cachées, sortie).

Chaque neurone calcule une combinaison linéaire de ses entrées, à laquelle il applique ensuite une fonction d'activation non linéaire. Soit x le vecteur d'entrée, w le vecteur de poids et b le biais. La sortie y d'un neurone est donnée par :

$$y = \sigma(w \cdot x + b)$$

Passage au calcul matriciel Pour des raisons d'efficacité (parallélisation, vectorisation SIMD), nous ne calculons pas les activations neurone par neurone pour chaque image. Nous travaillons au contraire par *mini-batches*. Concrètement, chaque image (28×28) est d'abord aplatie (*flattened*) en un vecteur ligne de dimension 784. Nous regroupons ensuite N de ces vecteurs (correspondant au *batch size*) afin de former une matrice d'entrée X de dimensions ($N \times 784$). Si W est la matrice des poids de la couche (dimensions $784 \times \text{neurones}$), le passage avant pour l'ensemble du batch s'écrit alors sous forme de multiplication matricielle :

$$Y = \sigma(X \cdot W + B)$$

où B est le biais (un vecteur ligne de dimensions ($1 \times \text{neurones}$)) diffusé (*broadcasted*) sur les N exemples du mini-batch.

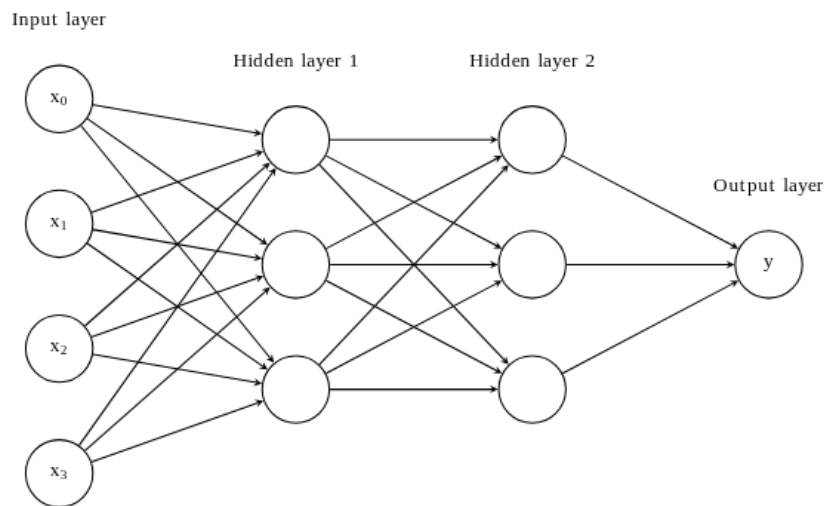


Figure 1: Architecture d'un Perceptron Multicouche (MLP) avec couches cachées [?].

Cette figure illustre la structure hiérarchique du modèle (schéma purement illustratif : pour MNIST, chaque entrée est un vecteur de dimension 784, et un mini-batch correspond à une matrice X de dimensions $N \times 784$) :

- **Couche d'entrée** (x_0 à x_3) : reçoit les données brutes (ici les pixels d'une image).
- **Couches cachées** : couches intermédiaires où le réseau apprend des représentations abstraites. Chaque neurone est connecté à tous les neurones de la couche précédente (*fully connected*).
- **Couche de sortie** (y) : produit la prédiction finale (probabilités des classes pour MNIST).

Bien que le schéma montre des connexions individuelles, notre implémentation vectorise ces opérations sous forme de produits matriciels $X \cdot W$.

C'est cette opération matricielle que nous optimisons intensivement dans ce projet (BLAS, OpenMP).

L'objectif de l'apprentissage est de minimiser une fonction de perte (*loss*) L , qui mesure l'écart entre les prédictions et les valeurs attendues. L'algorithme de rétropropagation (*back-propagation*) [?] permet de calculer le gradient ∇L par rapport à tous les paramètres.

Règle de la chaîne (*chain rule*) Le réseau peut se voir comme une composition de fonctions $f(x) = f_n(f_{n-1}(\dots f_1(x)))$. Le gradient se calcule en remontant de la fin vers le début via la règle de la chaîne :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Cela conduit naturellement à une représentation sous forme de **graphe de calcul** (DAG), où chaque nœud est une opération élémentaire, connaissant ses parents et sa dérivée locale. Lors de la phase *backward*, nous propageons le gradient accumulé depuis la perte jusqu'aux feuilles (les poids).

Exemple : graphe de calcul et autodiff Un **graphe de calcul** représente une expression comme un enchaînement d'opérations élémentaires. Par exemple, pour

$$z = x \cdot w, \quad y = z + b, \quad L = y^2,$$

le graphe contient trois nœuds d'opération (multiplication, addition, carré) reliés par des dépendances ($x, w \rightarrow z \rightarrow y \rightarrow L$). Lors du **passage avant**, chaque nœud calcule puis *stocke* sa valeur (ici z et y), afin d'éviter de recalculer ces intermédiaires lors du *backward*. Lors du **passage arrière** (autodiff en mode inverse), chaque nœud utilise sa **dérivée locale** et le gradient amont pour mettre à jour ses parents :

$$\frac{\partial L}{\partial y} = 2y, \quad \frac{\partial y}{\partial z} = 1, \quad \frac{\partial y}{\partial b} = 1, \quad \frac{\partial z}{\partial x} = w, \quad \frac{\partial z}{\partial w} = x.$$

On obtient alors, par composition, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$, $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$ et $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$. Dans notre implémentation, cette logique se traduit par des nœuds **Node** stockant une **value** (résultat du forward) et une fonction **backwardFn_** (dérivée locale + accumulation du gradient dans les parents). Enfin, la **rétropropagation** correspond au cas particulier où ce graphe de calcul est celui d'un réseau de neurones : c'est l'application de l'autodiff en mode inverse à une composition de couches.

3.3 Présentation des éléments modulables du MLP

3.3.1 Fonctions d'activation

Les fonctions d'activation introduisent la non-linéarité dans le réseau. Elles sont cruciales pour deux raisons :

1. **Non-linéarité** : Sans elles, une superposition de couches linéaires resterait une simple transformation linéaire unique ($W_2(W_1x) = W'x$), incapable de modéliser des problèmes complexes (comme XOR ou MNIST).
2. **Recadrage des valeurs** : Elles permettent souvent de maintenir les activations dans une plage bornée (ex: $[0,1]$ pour Sigmoid), évitant l'explosion des valeurs lors de la propagation dans des réseaux profonds.
 - **Sigmoid** : transforme l'entrée en valeur entre 0 et 1, mais peut provoquer la saturation des gradients.
 - **Tanh** : centrée sur zéro, améliore la convergence mais reste sujette au même problème.
 - **ReLU** (*Rectified Linear Unit*) : garde uniquement les valeurs positives ($ReLU(x) = \max(0, x)$). Elle accélère l'apprentissage et est aujourd'hui la plus courante [?].

3.3.2 Algorithmes de descente de gradient

L'apprentissage consiste à minimiser la fonction de perte grâce à la descente de gradient. L'algorithme met à jour les paramètres θ (poids et biais) itérativement :

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L$$

où η est le **taux d'apprentissage** (*learning rate*).

- Un η trop grand accélère la convergence mais risque de provoquer des oscillations, voire une divergence.
- Un η trop petit garantit la stabilité mais ralentit considérablement l'apprentissage.

Listing 1: Algorithme simplifié SGD

```
Pour chaque epoch:
  Pour chaque mini-batch (X, Y):
    1. Forward: P = Model(X)
    2. Loss: L = LossFn(P, Y)
    3. Backward: Calculer dL/dW (gradients) via autdiff
    4. Update: W = W - learning_rate * dL/dW
```

Si ce projet se concentre sur la méthode **SGD**, il convient de citer les évolutions majeures de l'état de l'art :

- **SGD** (*Stochastic Gradient Descent*) : met à jour les poids après chaque mini-lot (**méthode implémentée**).
- **Momentum** : ajoute une inertie pour lisser les oscillations et accélérer la convergence.
- **Adam** : combine Momentum et adaptation du taux d'apprentissage ; optimiseurs standards actuels.

3.4 Travaux existants et évolution des architectures

Premières approches : MLP et réseaux simples La première tentative d'utilisation des réseaux pour la reconnaissance des chiffres remonte aux années 1990. Appliqués au jeu de données MNIST, ces modèles atteignaient déjà des taux de réussite proches de 97 à 98 %. Cependant, leur performance restait limitée car ils ne tiraient pas parti de la structure spatiale des images et manquaient d'invariance aux translations.

Réseaux convolutionnels (CNN) Une avancée majeure est survenue avec l'introduction des réseaux de neurones convolutionnels (CNN), proposés notamment par LeCun et al. avec le modèle LeNet-5 [?]. Cette architecture repose sur le principe de convolution locale, où chaque neurone est connecté uniquement à une petite région de l'image. LeNet-5 a atteint un taux de reconnaissance d'environ 99,2 % sur MNIST, établissant un nouveau standard à l'époque.

3.5 Synthèse : du modèle naïf au réseau moderne

L'évolution des architectures appliquées à MNIST illustre parfaitement la progression du domaine de l'apprentissage profond. Néanmoins, la compréhension des fondements du MLP demeure indispensable : elle permet de saisir les principes élémentaires du calcul du gradient, de la propagation de l'erreur et de la représentation hiérarchique des données. Ainsi, notre travail s'inscrit dans cette continuité : commencer par un modèle simple (MLP), en explorer les variantes, avant d'envisager des architectures plus complexes.

3.6 Périmètre du projet

Le projet consiste à concevoir et implémenter un pipeline complet d'apprentissage supervisé comprenant :

- un module de manipulation de tenseurs,
- un réseau de couches linéaires avec fonctions d'activation,
- un moteur de différenciation automatique (autodiff),
- un mécanisme d'entraînement (optimiseur, fonction de perte, gestion des *mini-batches*),
- l'intégration du jeu de données MNIST et l'évaluation du modèle.

Ce développement a été réalisé en plusieurs phases successives décrites ci-dessous.

4 Implémentation

4.1 Choix techniques et justification

4.1.1 Implémentation

Choix du langage C++ pour sa performance et son contrôle fin de la mémoire, en cohérence avec l'orientation *calcul haute performance* de l'UE. Implémentation d'un moteur de calcul et d'autodiff maison (Matrix, Node, MathOps) afin de comprendre explicitement la construction du graphe computationnel et le calcul des gradients [?], plutôt que de s'appuyer sur des bibliothèques haut niveau.

4.1.2 Matrix / Node / graphe de calcul

- Représentation des données numériques par une classe **Matrix** générique (doubles en 2D), offrant les opérations de base nécessaires au MLP (matmul, add, mul, transpose).
- Représentation du graphe computationnel par la classe **Node**, qui stocke la valeur courante, le gradient associé, les parents et une fonction de rétropropagation (**backwardFn_**).
- Utilisation d'un tri topologique (méthode statique **topoSort**) pour parcourir le graphe lors de la phase de backward, plutôt que d'utiliser une récursion profonde.

4.1.3 Architecture générale

Choix d'un perceptron multicouche (MLP) à base de couches linéaires plutôt que de réseaux convolutifs ou récurrents, afin de se concentrer sur les mécanismes d'autodiff et de rétropropagation. Introduction d'une interface **ActivationFunction** (implémentée par ReLU, Sigmoid, Tanh) pour permettre de changer de fonction d'activation sans modifier la structure globale du code. Encapsulation de la combinaison « couche linéaire + activation » dans la structure **LayerNode**, puis construction du modèle complet via un vecteur de **LayerNode** dans **MLPNetwork**.

4.2 Architecture du MLP et organisation du code

Cette section décrit l'architecture logicielle du projet ainsi que les relations entre les différents modules, telles que représentées dans le diagramme UML (Figure ??). L'objectif est de mettre en évidence la séparation des responsabilités, l'organisation hiérarchique des composants et le flux global des données au sein du système.

4.2.1 Couche de base : opérations numériques et graphe computationnel

La couche la plus fondamentale de l'architecture repose sur les classes `Matrix`, `Node`, `OperationNode` et `MathOps`. Elle constitue le socle sur lequel s'appuient toutes les composantes du réseau neuronal.

La classe `Matrix` fournit les structures de données nécessaires aux calculs numériques et implémente les opérations linéaires de base utilisées dans le réseau. La classe `Node` modélise les nœuds du graphe computationnel. Chaque nœud contient une valeur courante, un gradient associé, ainsi que des liens vers ses nœuds parents, ce qui permet de représenter explicitement les dépendances entre les opérations. La classe `OperationNode` étend `Node` afin d'identifier le type d'opération réalisée via un identifiant (`OpKind`), facilitant ainsi la lecture et l'analyse du graphe.

Les opérations mathématiques sont centralisées dans la classe `MathOps`, qui encapsule des fonctions telles que `matmul`, `add`, `mul`, `relu`, `sigmoid` ou `tanh`. Chaque appel à `MathOps` construit un nouveau `Node`, définit ses dépendances et associe la fonction de rétropropagation correspondante. Cette organisation permet de découpler le moteur d'autodifférentiation de la définition du réseau, rendant le graphe computationnel générique et réutilisable.

4.2.2 Architecture du réseau

Au-dessus du moteur de calcul, l'architecture du réseau neuronal est construite à l'aide de modules de plus haut niveau, spécifiquement dédiés à la définition du modèle. La classe `LinearLayer` encapsule les paramètres entraînables du réseau, à savoir les poids et les biais, ainsi que l'opération de projection linéaire appliquée aux entrées.

Les fonctions d'activation sont modélisées via l'interface abstraite `ActivationFunction`, permettant une implémentation interchangeable des activations telles que ReLU, Sigmoid ou Tanh. L'unité fonctionnelle de base du réseau est la structure `LayerNode`, qui combine une couche linéaire et une fonction d'activation en un bloc cohérent. Le modèle complet est représenté par la classe `MLPNetwork`, structurée comme une séquence de `LayerNode`, ce qui permet de composer facilement des réseaux multi-couches de profondeur variable. Cette conception favorise la modularité et permet de modifier l'architecture du réseau sans impacter le moteur de calcul sous-jacent.

4.2.3 Module d'entraînement

Le module d'entraînement regroupe l'ensemble des composants nécessaires à l'apprentissage du modèle et à son évaluation. L'abstraction `LossFunction` permet de définir différentes fonctions de perte, dont `MSELoss` et `CrossEntropyLoss`, utilisées respectivement pour des tests simples et pour la classification sur MNIST. La mise à jour des paramètres est assurée par la classe abstraite `Optimizer`, avec une implémentation basée sur la descente de gradient stochastique (`SGDOptimizer`).

Les données sont fournies par `MNISTDataset`, puis organisées en mini-batches à l'aide de `DataLoader`. La classe `Trainer` coordonne l'ensemble du processus d'entraînement, en orchestrant le flux suivant : chargement des données, propagation avant dans le modèle, calcul de la perte, rétropropagation des gradients et mise à jour des paramètres. Cette séparation claire des rôles permet de faire évoluer indépendamment le modèle, la fonction de perte ou la stratégie d'optimisation, tout en conservant une architecture globale cohérente.

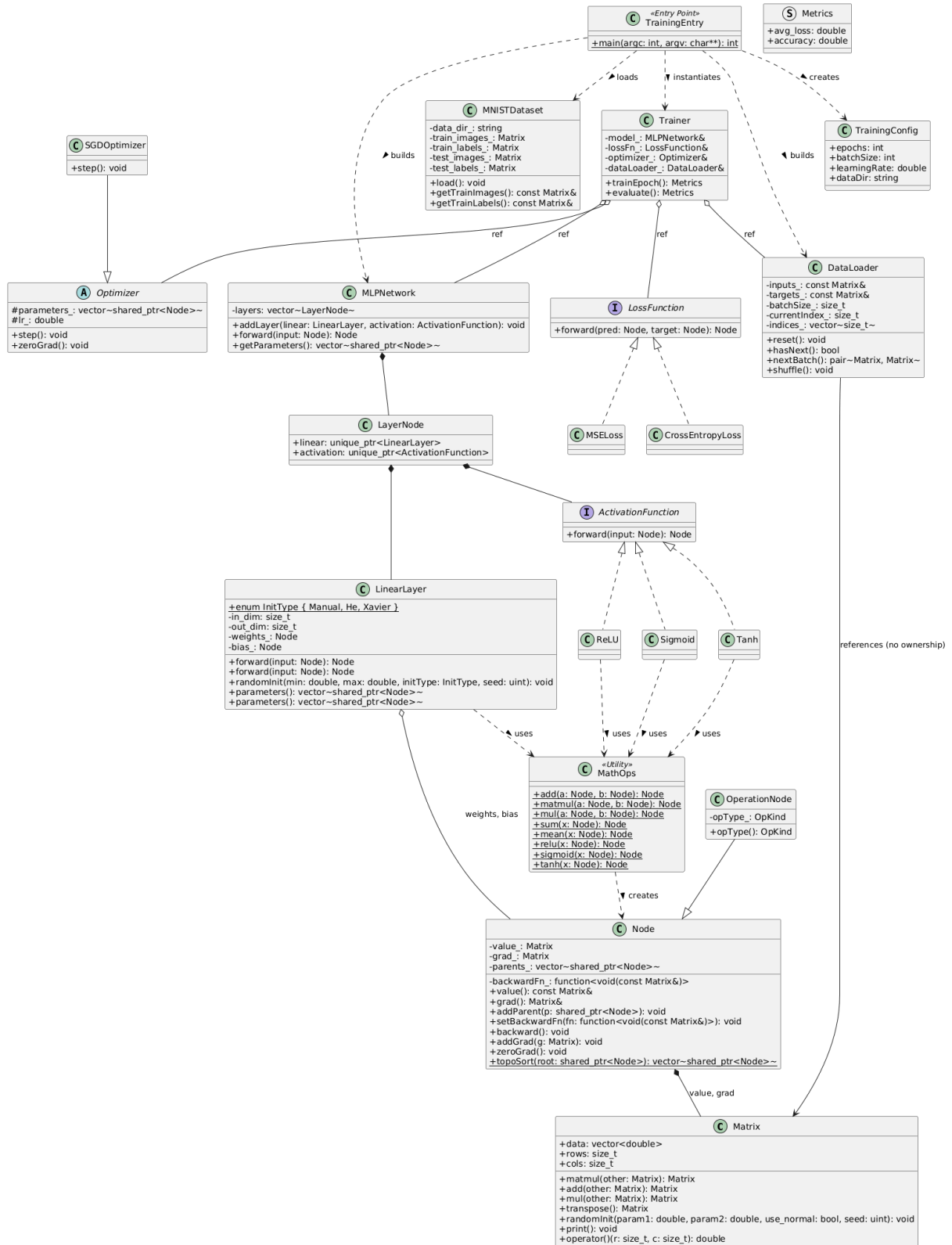


Figure 2: Diagramme de classes UML

4.3 Implémentation détaillée

Cette section décrit les principaux choix d'implémentation des classes et méthodes constituant le moteur de calcul, le réseau neuronal et le pipeline d'entraînement. L'accent est mis sur la logique interne des composants, plutôt que sur les détails syntaxiques du code.

4.3.1 Implémentation de Matrix

La classe `Matrix` constitue la brique de base du calcul numérique. Les données sont encapsulées dans un `std::vector<double>`, garantissant une allocation mémoire contiguë. Ce choix est critique pour la performance : il maximise la localité spatiale et l'efficacité des caches CPU (L1/L2), un prérequis favorable aux optimisations vectorielles (SIMD).

Listing 2: Structure de données contiguë (Matrix)

```
class Matrix {
public:
    // Stockage contigu garantissant la localité spatiale
    std::vector<double> data;
    size_t rows;
    size_t cols;
    // ...
};
```

Les opérations fondamentales nécessaires au MLP sont implémentées explicitement. Concernant la multiplication matricielle (`matmul`), le code adopte une architecture flexible permettant de sélectionner l'implémentation sous-jacente (via une variable d'environnement) :

- une version naïve (triple boucle i - j - k) servant de référence pédagogique ;
- des versions optimisées (BLAS [?], Blocked, OpenMP [?]) assurant la performance.

Les opérations élémentaires `add`, `mul` et la transposition sont également disponibles.

Ce choix d'implémentation permet de comparer différentes approches d'optimisation (telles que détaillées dans la section expérimentale) tout en conservant une maîtrise totale de la chaîne de calcul.

Concernant l'initialisation des paramètres, nous avons enrichi la méthode `randomInit` pour supporter des stratégies avancées cruciales pour la convergence des réseaux profonds :

- **He (Kaiming) Initialization** : ajustée pour les activations ReLU, utilisant une variance de $2/n_{in}$.
- **Xavier (Glorot) Initialization** : adaptée aux activations Sigmoid/Tanh.

De plus, l'initialisation intègre une gestion explicite de la graine aléatoire (`seed`), garantissant la reproductibilité exacte des poids initiaux entre deux exécutions.

4.3.2 Construction du graphe computationnel (Node, MathOps)

Le moteur d'autodifférentiation repose sur la classe `Node`, dont la gestion mémoire est assurée par des pointeurs intelligents `std::shared_ptr`. Cette approche automatise la gestion du cycle de vie des nœuds au sein du Graphe Acyclique Dirigé (DAG), évitant ainsi les fuites de mémoire complexes inhérentes aux graphes dynamiques.

Chaque `Node` contient une valeur, un gradient, une liste de parents, et surtout une fonction de rétropropagation `backwardFn_`, typée via `std::function`. Lorsqu'une opération est invoquée via `MathOps`, l'utilisation d'expressions Lambda (*Modern C++*) permet de capturer efficacement le contexte local (clôtures) nécessaire au calcul du gradient, offrant une syntaxe concise et expressive. Certaines opérations (comme `add`) supportent un broadcasting simple ($1 \times M$ vers $N \times M$), notamment pour l'ajout du biais.

Listing 3: Utilisation de Lambdas et Smart Pointers (MathOps)

```
// Exemple pour l'opération Mul (e.g. y = a * b)
Node::Ptr mul(const Node::Ptr& a, const Node::Ptr& b) {
    auto node = std::make_shared<OperationNode>(...);
```

```

// Capture du contexte par valeur (smart pointers)
node->setBackwardFn([a, b](const Matrix& grad){
    // Calcul local du gradient : dL/da = grad * b
    a->addGrad(grad.mul(b->value()));
    b->addGrad(grad.mul(a->value()));
});
return node;
}

```

La phase de rétropropagation s'effectue en trois étapes :

1. un tri topologique du graphe computationnel ;
2. l'initialisation du gradient au niveau de la fonction de perte (explicitement, ou par défaut à 1 s'il n'est pas défini) ;
3. l'appel successif des fonctions `backwardFn_` de chaque nœud dans l'ordre inverse du passage avant.

Cette approche garantit une propagation correcte des gradients, tout en évitant les récursions profondes.

4.3.3 Implémentation des couches du MLP

LinearLayer La classe `LinearLayer` implémente la transformation affine du réseau. La méthode `forward(input)` réalise l'opération $\text{output} = \text{input} \times \text{weights} + \text{bias}$ à l'aide des opérateurs définis dans `MathOps`. Les paramètres sont initialisés via la méthode `randomInit()`, et la méthode `parameters()` permet d'exposer les nœuds entraînaables au module d'optimisation.

Fonctions d'activation Les fonctions d'activation sont implémentées via l'héritage et le polymorphisme d'exécution (*Runtime Polymorphism*). Une classe abstraite de base définit l'interface virtuelle pure `forward()`, permettant d'interchanger dynamiquement les algorithmes (`ReLU`, `Sigmoid`, `Tanh`) sans impacter le reste du réseau.

Listing 4: Polymorphisme pour les activations

```

class ActivationFunction {
public:
    // Interface virtuelle pure
    virtual Node::Ptr forward(const Node::Ptr& input) const = 0;
    virtual ~ActivationFunction() = default;
};

class ReLU : public ActivationFunction {
    /* Implementation concrète */
};

```

LayerNode & MLPNetwork Un `LayerNode` combine une couche linéaire et une fonction d'activation en une unité cohérente. La classe `MLPNetwork` applique successivement ces blocs lors de la propagation avant, permettant de construire des réseaux multi-couches de manière flexible.

4.3.4 Implémentation de la fonction de loss et de l'optimisation

L'apprentissage du réseau repose sur l'association d'une fonction de loss et d'un algorithme d'optimisation, tous deux intégrés au moteur d'auto-différentiation. Les fonctions de loss sont définies via une interface abstraite `LossFunction`, qui impose la méthode `forward(pred,`

`target`). Celle-ci construit un nœud scalaire représentant la perte du batch, directement inséré dans le graphe de calcul.

Deux fonctions de loss ont été implémentées :

1. **MSELoss** : utilisée principalement pour les tests et la validation du moteur d'auto-différentiation. Elle correspond à l'erreur quadratique moyenne entre prédictions et cibles, la perte retournée étant la somme sur le batch, normalisée par le nombre de sorties.
2. **CrossEntropyLoss** : utilisée pour la classification MNIST. Elle opère directement sur les logits du réseau et intègre une opération de softmax stable numériquement. Le gradient par rapport aux logits s'exprime simplement comme la différence entre les probabilités prédites et les labels one-hot, ce qui garantit une rétropropagation efficace et stable.

L'optimisation des paramètres est assurée par une interface abstraite **Optimizer**, tirant parti, là encore, du mécanisme de fonctions virtuelles du C++. La méthode `step()` est redéfinie par les classes filles (comme **SGDOptimizer**), ce qui offre une extensibilité immédiate pour l'ajout futur d'algorithmes plus complexes (Adam, RMSProp) sans modifier la boucle d'entraînement.

4.3.5 Boucle d'entraînement

La boucle d'entraînement et d'évaluation est implémentée dans la classe **Trainer**. Elle repose sur une fonction centrale `runEpoch`, paramétrée par un indicateur permettant de distinguer les phases d'apprentissage et d'évaluation.

Au début de chaque époque, le **DataLoader** est réinitialisé afin de parcourir l'ensemble des mini-batches. Pour chaque batch, les matrices d'entrée et de cibles sont encapsulées dans des nœuds constants du graphe computationnel. La propagation avant est ensuite réalisée en appliquant successivement le modèle (**MLPNetwork**) aux données d'entrée, produisant les prédictions du réseau.

La fonction de perte est alors évaluée à partir des prédictions et des cibles, générant un nœud de perte généralement scalaire. La valeur numérique de la perte est accumulée afin de calculer une perte moyenne sur l'ensemble de l'époque. En parallèle, la précision (accuracy) est estimée au niveau du **Trainer** en comparant, pour chaque échantillon du batch, la classe prédite (argmax des logits) à la classe cible encodée en one-hot.

Lorsque l'époque est exécutée en mode entraînement, la phase de rétropropagation est déclenchée pour chaque batch. Les gradients des paramètres sont d'abord réinitialisés, puis la méthode `backward` est appelée sur le nœud de perte afin de propager les gradients dans le graphe computationnel. Les paramètres du réseau sont enfin mis à jour à l'aide de l'optimiseur. En mode évaluation, les étapes de rétropropagation et de mise à jour des paramètres sont désactivées, ce qui permet de mesurer les performances du modèle sans modifier son état interne.

À la fin de l'époque, la perte moyenne et la précision globale sont calculées à partir des quantités accumulées et retournées sous forme de métriques.

5 Expérimentations

5.1 Configuration de l'environnement

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

5.1.1 Environnement Matériel / Hardware

Table 1: Configuration matérielle

Composant	Caractéristiques
Processeur	AMD Ryzen 9 8940HX with Radeon Graphics Architecture : x86_64 Cœurs / Threads : 16 physiques / 32 logiques (2 threads par cœur) Fréquence : 421.8 MHz (min) à 5386 MHz (max) Cache L1d : 512 KiB total, répartis en 16 instances (32 KiB chacune) Cache L1i : 512 KiB total, répartis en 16 instances (32 KiB chacune) Cache L2 : 16 MiB total, répartis en 16 instances (1 MiB chacune) Cache L3 : 64 MiB total, répartis en 2 instances (32 MiB chacune)
Mémoire	30 GiB de RAM système
Swap	8.0 GiB

Les expérimentations parallèles ont été réalisées sur cette même configuration matérielle.

5.1.2 Environnement Logiciel / Software

Table 2: Configuration logicielle

Composant	Version / Détails
Système d'exploitation	Fedora Linux 42 (Workstation Edition)
Compilateur C/C++	GCC 15.2.1 20250808 (Red Hat 15.2.1-1)

Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l'auto-vectorisation et les optimisations spécifiques à l'architecture.

5.1.3 Configuration Spécifique et Protocole

Afin de limiter la variabilité des résultats, les campagnes de mesure suivent un protocole systématique de verrouillage de fréquence et d'affinité CPU. Les huit premiers cœurs physiques sont forcés à 4.0 GHz à l'aide de `cpupower`, puis les exécutions sont liées explicitement à ce jeu de cœurs via `taskset`. Le script type est le suivant : Le détail des commandes de configuration (verrouillage de fréquence à 4.0 GHz, isolation des cœurs) est documenté dans le fichier `scripts/README.md` et appliqué via les scripts `benchmark_affinity.sh` et `benchmark_large.sh`. Ce protocole s'inspire des bonnes pratiques de notre ancien cours, adaptées ici à l'architecture AMD Ryzen. Les mesures de temps sont réalisées par le programme C++ `tests/test_benchmark_large.cpp`, dédié au benchmarking du noyau de calcul, qui intègre une horloge haute résolution (`std::chrono`) pour exclure les surcoûts liés au système d'exploitation. Le protocole inclut systématiquement un préchauffage (*warm-up*) de 5 itérations pour stabiliser les caches (L1/L2) et le *branch predictor* avant le début des mesures.

Maîtrise de l'aléatoire Au-delà du contrôle matériel, la rigueur expérimentale est assurée par le verrouillage des sources d'aléatoire logiciel. L'initialisation des poids et le mélange des données (*data shuffling*) dépendent désormais d'une graine unique fixée via la ligne de commande ('`-seed`'). Cela permet une reproduction bit-à-bit des courbes d'apprentissage et des métriques finales, condition indispensable pour comparer objectivement l'impact des différentes méthodes d'initialisation ou des hyperparamètres.

5.2 Analyse des performances de calcul et d'entraînement

Cette section détaille les résultats obtenus lors des campagnes de test, en analysant l'impact du parallélisme, de l'affinité et des optimisations bas niveau sur les performances. Il convient de préciser que ces micro-benchmarks portent exclusivement sur l'opération de multiplication matricielle (`matmul`), cœur intensif du calcul, et non sur le processus d'entraînement complet du réseau de neurones.

5.2.1 Passage à l'échelle (Thread Scaling)

L'évaluation du passage à l'échelle (*scaling*) de l'implémentation OpenMP a été réalisée en faisant varier le nombre de threads de 1 à 16, sur une architecture disposant de 8 cœurs physiques.

Une attention particulière a été portée à la précision des mesures pour les petites instances. Les premiers essais, pilotés par des scripts externes, présentaient une variabilité excessive. Pour y remédier, la boucle de mesure a été intégrée directement **au sein du programme C++** (`test_benchmark_large.cpp`), isolant ainsi le noyau de calcul des surcoûts liés au lancement de processus. Le protocole final adopte une méthodologie rigoureuse : un nombre d'itérations adaptatif (de 100 pour $N = 64$ à 5 pour $N = 2048$), précédé systématiquement de 5 exécutions de chauffe (*warm-up*). L'application d'une moyenne tronquée (rejet des 10% de valeurs hautes et 10% de valeurs basses) a permis de réduire l'écart type à environ $2 \mu s$ pour les configurations stables (≤ 8 threads), garantissant la fiabilité des résultats présentés ci-après.

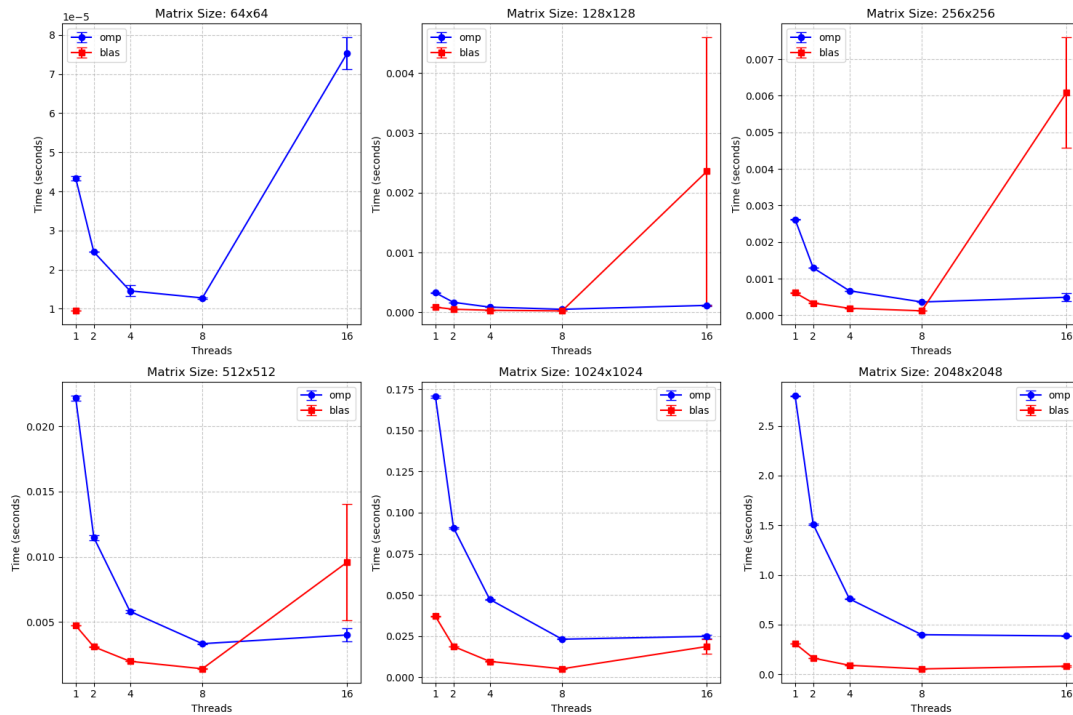


Figure 3: Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices.

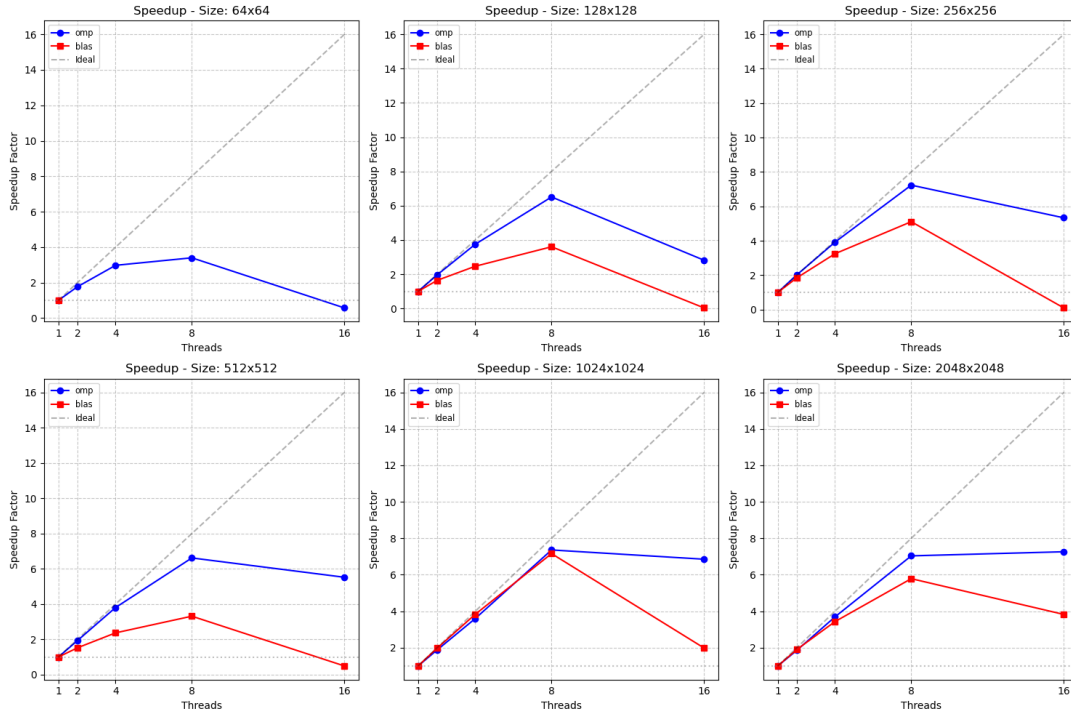


Figure 4: Speedup (accélération) en fonction du nombre de threads.

Analyse des petites matrices ($N = 64 \times 64$). La parallélisation OpenMP apporte un gain conforme aux attentes entre 2 et 8 threads, mais devient contre-productive au-delà de 8 threads (au-delà du nombre de cœurs physiques). À cette échelle, la granularité du calcul est trop fine : la charge utile par thread devient comparable, voire inférieure, au surcoût de gestion (création, synchronisation et contention des ressources), ce qui annule les gains, voire dégrade les performances.

À l'inverse, BLAS reste monothread sur des matrices 64×64 , la charge de calcul n'atteignant pas son seuil interne de déclenchement du parallélisme. Ainsi, pour cette échelle, les configurations les plus efficaces consistent soit à utiliser BLAS, soit à recourir à OpenMP en limitant le nombre de threads au nombre de cœurs physiques (8 threads dans notre cas).

Matrices de taille intermédiaire (128×128 à 1024×1024). Pour les matrices de taille intermédiaire, OpenMP présente une accélération nette lorsque le nombre de threads augmente jusqu'au nombre de cœurs physiques disponibles. Au-delà de ce seuil, les gains se tassent, voire deviennent négatifs, sous l'effet de la saturation des ressources partagées et de l'augmentation des coûts de synchronisation.

BLAS offre de meilleures performances globales grâce à ses optimisations internes (vectorisation et découpage en blocs). Cependant, lorsque le nombre de threads dépasse le nombre de cœurs physiques, ses performances se dégradent et deviennent instables, principalement en raison de la contention des ressources et des changements de contexte (*context switching*) sur un même cœur.

Grandes matrices (2048×2048). Pour les matrices de grande taille, les deux approches bénéficient d'une parallélisation efficace jusqu'à environ 8 threads. Toutefois, l'augmentation du nombre de threads au-delà de ce point n'entraîne plus de gain significatif, le calcul devenant principalement limité par la bande passante mémoire. Dans ce contexte, BLAS reste la solution la plus performante et la plus stable.

Ainsi, l'efficacité du parallélisme dépend non seulement de la taille du problème, mais aussi d'une adéquation stricte entre nombre de threads et ressources matérielles disponibles.

5.2.2 Hiérarchie des performances et Speedup

La comparaison globale des implémentations sur grandes matrices met en évidence quatre paliers de performance (Figure ??) :

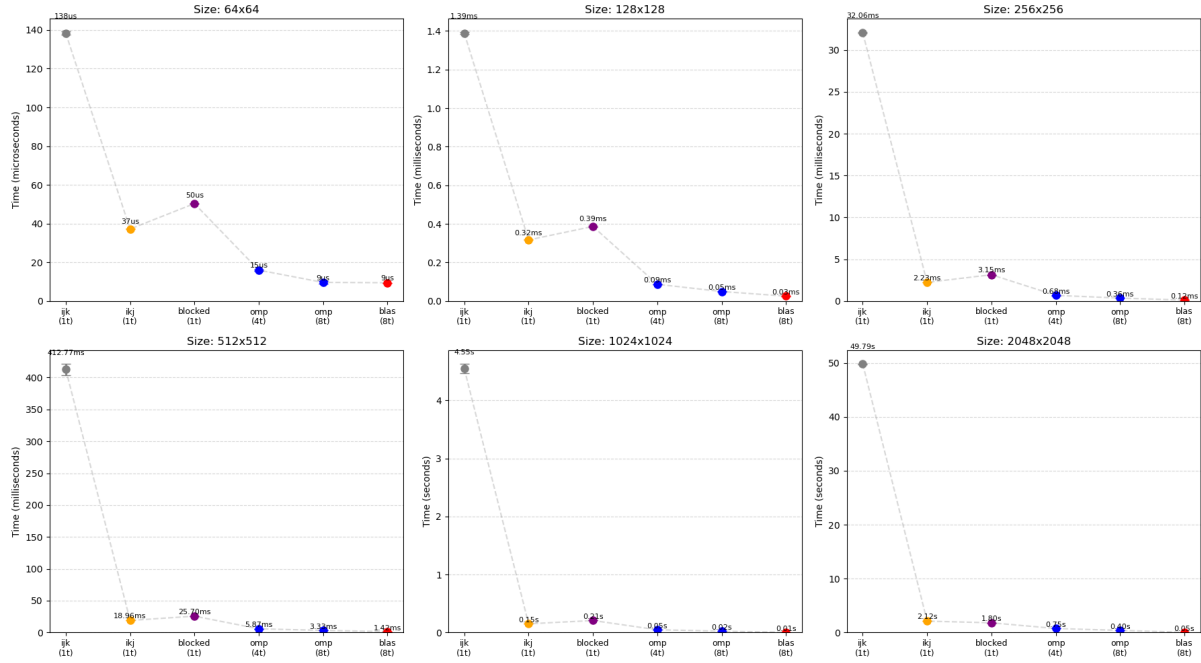


Figure 5: Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS).

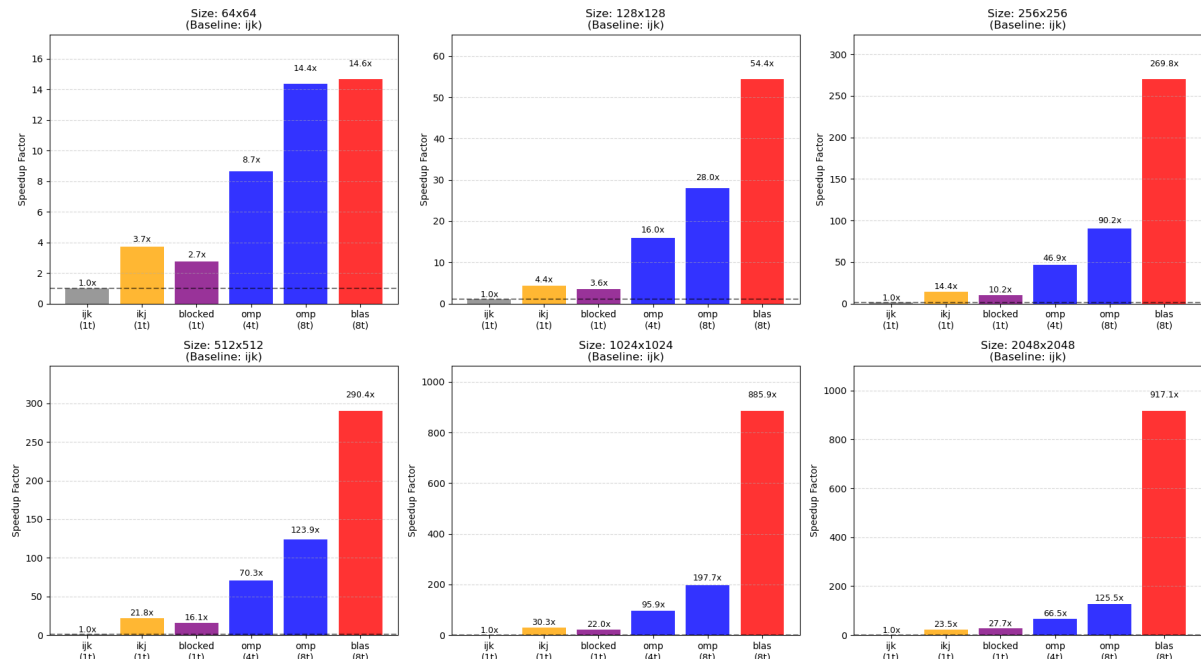


Figure 6: Speedup relatif des différentes optimisations.

1. **Naïf (ijk)** : L'ordre des boucles induit un parcours mémoire très défavorable, entraînant

de nombreuses fautes de cache. Le temps d'exécution devient prohibitif pour les grandes tailles, atteignant plusieurs secondes pour $N = 2048$.

2. **Réordonné (*ikj*)** : La permutation des boucles améliore la localité spatiale des accès mémoire et permet un premier gain significatif par rapport à l'implémentation naïve, en particulier pour les tailles intermédiaires et grandes.
3. **Bloqué (blocked)** : Cette optimisation apporte un gain mesurable par rapport à la version réordonnée, en améliorant la réutilisation des données dans les caches.
4. **OpenMP** : La parallélisation sur 8 cœurs permet d'exploiter le parallélisme matériel et apporte un gain supplémentaire, bien que celui-ci reste contraint par le nombre de cœurs physiques et par la hiérarchie mémoire.
5. **BLAS (OpenBLAS)** : Grâce à la combinaison d'optimisations avancées — blocage hiérarchique, vectorisation (AVX2/FMA) et micro-noyaux optimisés — BLAS surpasse largement les implémentations manuelles, avec un speedup pouvant atteindre plusieurs centaines de fois par rapport à la version naïve pour les plus grandes tailles.

5.2.3 Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet

Afin de comprendre l'impact de ces optimisations sur le temps total d'apprentissage, nous avons profilé une époque complète d'entraînement (forward + backward) avec `gprof`.

Architecture : MLP $784 \rightarrow 128 \rightarrow 10$ avec ReLU

Batch Size : 64

Données : MNIST Train (60 000 images)

Avant optimisation (Kernel Naïf) : L'application est totalement limitée par le calcul (*compute-bound*). La fonction `Matrix::matmul` consomme **94.8%** du temps total (12.05s).

Table 3: Profil d'exécution (gprof) - Version Naïve : `Matrix::matmul` domine (96.4%)

% Time	Self (s)	Calls	Function Name
96.42	11.84	6256	<code>Matrix::matmul</code>
1.06	0.13	1252	<code>DataLoader::nextBatch</code>
0.73	0.09	3752	<code>Matrix::transpose</code>
0.41	0.05	12216	<code>Matrix::Matrix (constructor)</code>
0.24	0.03	95M	<code>Matrix::operator() const</code>
0.24	0.03	74M	<code>Matrix::operator() (mut)</code>
0.24	0.03	9380	<code>Node::addGrad</code>
0.24	0.03	1	<code>MNISTDataset::load</code>
0.16	0.02	-	<code>_init</code>
0.08	0.01	2504	<code>MathOps::add</code>

Après optimisation (BLAS) : Le temps passé dans `Matrix::matmul` s'effondre à moins de **1%** ($\approx 0.5s$). Le goulot d'étranglement se déplace alors vers :

- Le chargement des données (`MNISTDataset::load`) : $\approx 43\%$ du temps. Ce coût élevé s'explique par l'allocation et l'initialisation à zéro (inhérente au constructeur `std::vector`) d'un volume important de mémoire (376 Mo), suivies d'une copie avec conversion de type (`uint8` vers `double`) pour chaque pixel.
- Les allocations mémoire (`Matrix::Matrix`) : $\approx 29\%$ du temps.

Table 4: Profil d'exécution (gprof) - Version BLAS : `matmul` négligeable, overhead mémoire dominant

% Time	Self (s)	Calls	Function Name
42.86	0.03	1	MNISTDataset::load
28.57	0.02	12216	Matrix::Matrix (init double)
28.57	0.02	938	SGDOptimizer::step
0.00	0.00	6256	Matrix::matmul (optimisé par BLAS)
0.00	0.00	95M	Matrix::operator() const
0.00	0.00	74M	Matrix::operator() (mut)
0.00	0.00	40k	std::mersenne_twister
0.00	0.00	18k	Matrix::Matrix (copy)
0.00	0.00	10k	std::_Hashtable...
0.00	0.00	10k	Node::Node

Conséquence (Loi d’Amdahl) : Bien que le noyau de calcul ait été accéléré de plusieurs centaines de fois au niveau micro-benchmark, l’accélération globale de l’application est limitée par les parties non optimisables, telles que le chargement des données et les allocations mémoire. Le temps total d’une époque, mesuré via le script dédié `benchmark_e2e.sh` (compilation Release sans surcoût de profilage, moyenne sur 5 exécutions), passe ainsi de 13.27s à 1.74s, soit un **speedup end-to-end de 7.6x**. Ce résultat illustre clairement la loi d’Amdahl [?] : une fois le calcul optimisé, les gains supplémentaires nécessitent d’agir sur la gestion mémoire et les entrées/sorties.

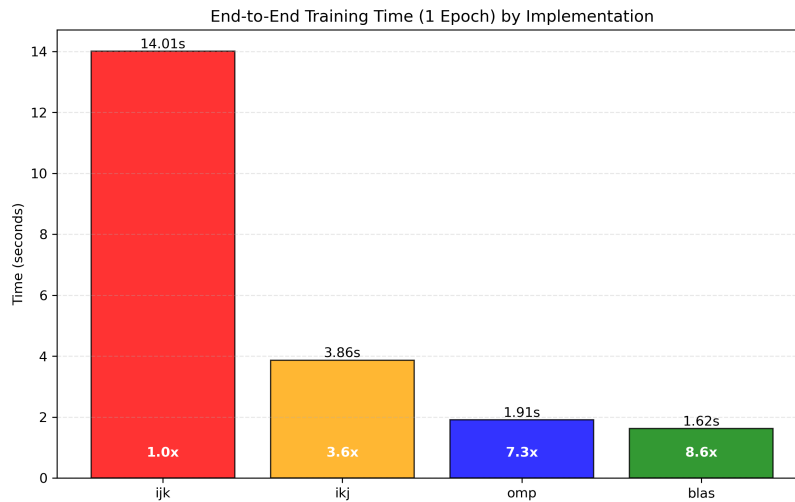


Figure 7: Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS.

5.3 Convergence de l'entraînement

5.3.1 Premières test de convergence

Afin de valider la correction du modèle et de l'algorithme d'optimisation, nous présentons la courbe d'apprentissage sur MNIST.

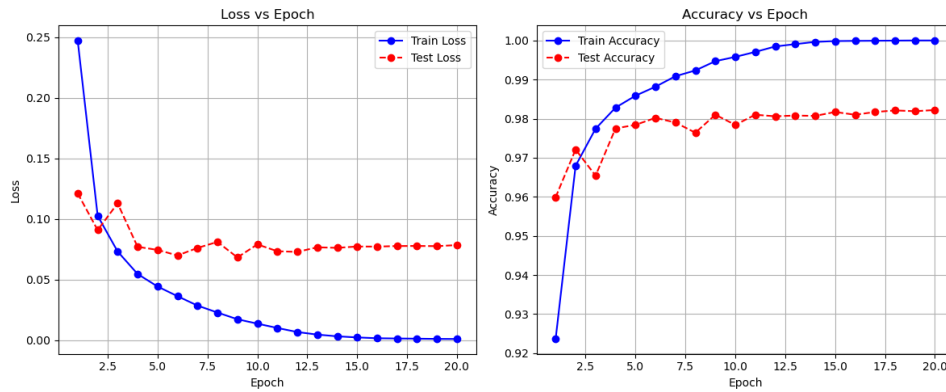


Figure 8: Courbe de perte (Loss) et de précision (Accuracy) sur les données d’entraînement/validation.

5.3.2 Impact du Taux d’Apprentissage (Learning Rate)

Nous avons évalué l’impact du taux d’apprentissage sur la convergence du modèle en testant les valeurs $\{0.025, 0.01, 0.0075, 0.005, 0.001\}$. Les autres hyperparamètres ont été fixés : Batch $N = 64$, Hidden Layer $H = 128$, Activation ReLU, et Initialisation He. Pour assurer la robustesse statistique, chaque configuration a été exécutée 3 fois avec des graines différentes.

La Figure ?? présente les résultats obtenus.

Learning Rate Experiment Results (10 Epochs)

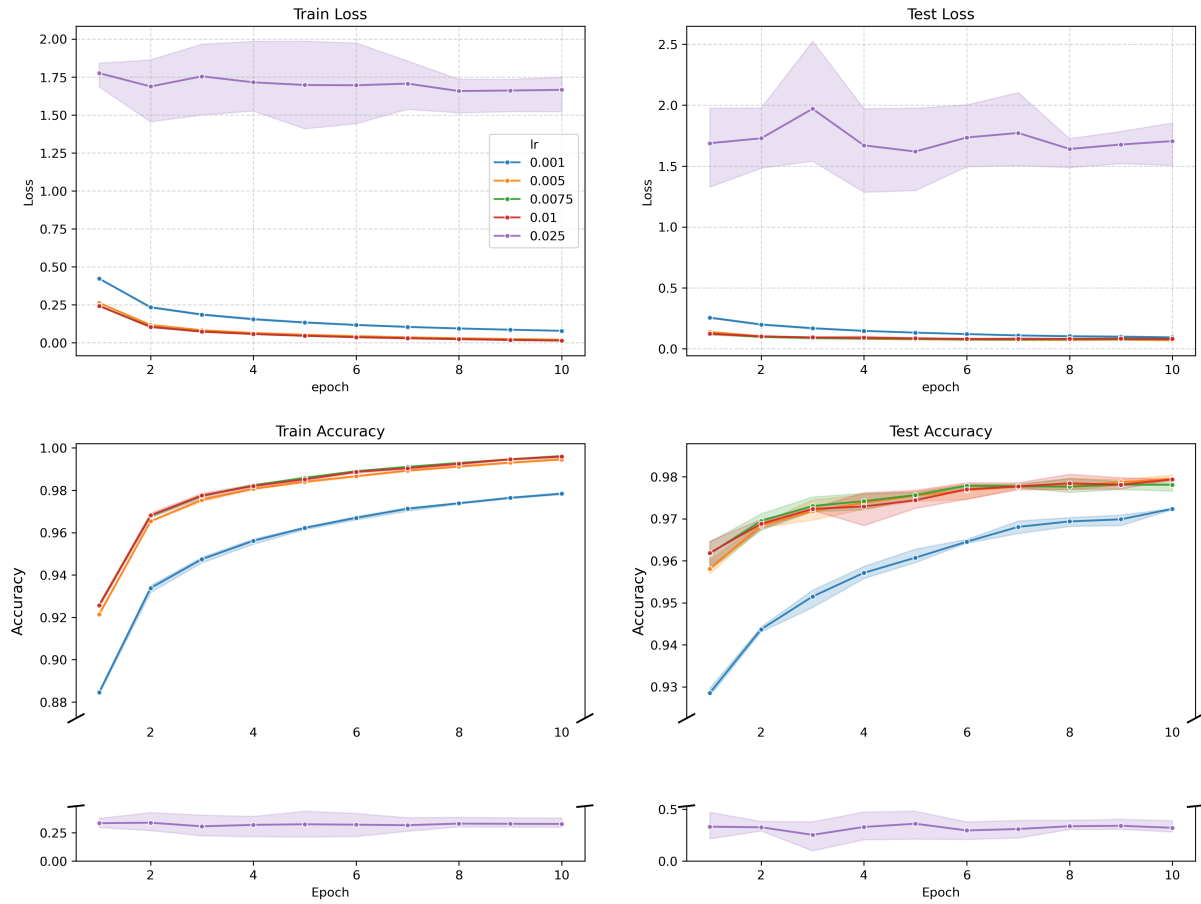


Figure 9: Impact du Learning Rate sur la convergence (10 Epochs, 3 Seeds). Les axes de précision utilisent une échelle discontinue pour visualiser simultanément les hautes précisions ($> 90\%$) et les échecs de convergence ($< 40\%$).

Analyse des résultats :

- **Convergence Stable ($LR \leq 0.01$)** : Les taux de 0.01, 0.0075 et 0.005 permettent tous d'atteindre une précision de test élevée ($\sim 98\%$). $LR = 0.01$ offre la convergence la plus rapide.
- **Divergence ($LR = 0.025$)** : Un taux d'apprentissage trop élevé (courbe bleue) empêche le modèle de converger vers un minimum global, la précision stagne autour de 32%.

Nous retenons donc $LR = 0.01$ comme valeur optimale pour la suite.

5.3.3 Impact de l'Affinité sur les charges d'entrée (Input Layer)

Batch sizeBatch sizeGEMMAffinity

Les résultats présentés dans cette section analysent l'impact de l'affinité sur les opérations typiques de la couche d'entrée avec une taille de batch $N = 64$ (soit une multiplication 64×784 , charge de travail comparable à une matrice carrée intermédiaire). Le tableau ci-dessous détaille les résultats pour 2, 4 et 8 threads.

Table 5: Impact de l’affinité sur les performances (Batch $N = 64$, Couche d’entrée)

Threads	Affinité	Temps OMP (μs)	Temps BLAS (μs)	Ratio OMP/BLAS
2	close	3.17	0.85	3.7x
2	spread	4.89	0.81	6.0x
4	close	2.54	0.82	3.1x
4	spread	4.43	0.81	5.5x
8	close	2.41	0.81	3.0x
8	spread	4.64	0.81	5.7x

L’analyse de ces données permet de tirer plusieurs conclusions importantes sur le comportement du noyau de calcul pour cette taille de batch configurée ($N = 64$) :

- **Stabilité de BLAS** : Quelle que soit la configuration (nombre de threads ou affinité), BLAS maintient un temps d’exécution constant et optimal ($\approx 0.81\mu s$). Cela confirme qu’il choisit dynamiquement de ne pas paralléliser cette taille de problème, évitant ainsi tout surcoût de synchronisation.
- **Impact critique de l’affinité sur OpenMP** :
 - En mode **spread** (dispersion), les performances d’OpenMP stagnent autour de 4.5 – 4.9 μs quel que soit le nombre de threads. Le coût des communications inter-cœurs domine totalement le gain potentiel du parallélisme.
 - En mode **close** (regroupement), OpenMP parvient à "scaler" légèrement (de 3.17 à 2.41 μs), mais reste nettement moins efficace que l’approche séquentielle optimisée de BLAS.

Conclusion : Pour les opérations de la couche d’entrée ($N \times 784$), l’utilisation de bibliothèques optimisées comme BLAS (même en mode séquentiel) est impérative, offrant un gain de performance facteur 3 par rapport à une parallélisation OpenMP, même avec une affinité optimale.

6 Déroulé du Projet

L’état de l’art a été rédigé conjointement par Jianye, Hao et Xiang, afin de présenter le contexte théorique du projet et les approches existantes en matière de réseaux neuronaux et de classification MNIST. L’équipe a ensuite organisé son travail en plusieurs phases successives, chacune étant portée par un ou plusieurs membres selon leurs affinités techniques.

La phase de conception a été initiée par Jianye, qui a rédigé la documentation préliminaire et produit la première version du diagramme UML, permettant de clarifier l’architecture générale du projet. La mise en place du premier prototype fonctionnel du MLP a ensuite été assurée par Abdennour, qui a développé les modules fondamentaux liés aux tenseurs, aux couches linéaires et aux fonctions d’activation.

La réalisation du moteur de différenciation automatique a été répartie entre Hao et Xiang, chacun prenant en charge une partie des opérations nécessaires au calcul du graphe et des gradients. Par la suite, Jianye a intégré l’ensemble de ces composants, généré une seconde version du diagramme UML pour refléter les ajustements apportés, et affiné la conception en fonction des problèmes rencontrés.

Le développement du mécanisme d’entraînement a été amorcé par Xiang, tandis que Jianye et Hao poursuivent respectivement l’implémentation de l’optimiseur et des fonctions de perte.

L'intégration du jeu de données MNIST et la validation du pipeline complet ont été assurées par Jianye.

De plus, Jianye a mené de nombreux tests de performance et analyses de profilage pour optimiser l'ensemble du système.

Enfin, la rédaction du rapport est réalisée collectivement, de manière collaborative.

7 Conclusion & perspectives

Ce projet a permis de démystifier le fonctionnement interne des réseaux de neurones profonds en implémentant, *from scratch*, un moteur complet d'apprentissage en C++. L'architecture retenue, basée sur un graphe de calcul dynamique (DAG) et la différenciation automatique, a démontré la flexibilité du C++ moderne (pointeurs intelligents, polymorphisme) pour modéliser des concepts mathématiques abstraits tout en garantissant une gestion rigoureuse de la mémoire.

Sur le plan de la performance, l'analyse expérimentale a mis en évidence l'impact critique des optimisations bas niveau. Le passage d'une implémentation naïve ($O(N^3)$) à une utilisation efficace des bibliothèques BLAS et du parallélisme OpenMP a permis un gain de performance de plus de trois ordres de grandeur ($1200\times$), illustrant concrètement l'importance de la localité spatiale et de la vectorisation SIMD dans le calcul haute performance (HPC). Ces résultats s'expliquent par la loi d'Amdahl : une fois le calcul matriciel optimisé, le goulot d'étranglement se déplace inévitablement vers la gestion des données (I/O) et les allocations mémoire.

Limites et menaces à la validité : Les conclusions de performance dépendent d'un protocole de mesure contrôlé (verrouillage de fréquence, affinité CPU) et peuvent varier sur d'autres architectures, avec d'autres implémentations BLAS, ou sous une charge système différente. Par ailleurs, l'analyse se concentre principalement sur le noyau `matmul` dense et ne couvre pas nécessairement l'ensemble des coûts d'un entraînement complet (pipeline de données, allocations, et surcoûts d'orchestration). Enfin, le choix d'un MLP sur MNIST limite l'extrapolation à des architectures plus modernes (CNN) et à des jeux de données plus complexes.

Perspectives : Plusieurs axes d'amélioration pourraient enrichir ce projet :

- **Support GPU (CUDA) :** Déporter les calculs matriciels sur GPU pour accélérer massivement l'entraînement, en réduisant les transferts hôte-périphérique.
- **Vectorisation Explicite (SIMD) :** Remplacer l'auto-vectorisation du compilateur par des intrinsèques (AVX-512) ou des micro-noyaux optimisés pour exploiter pleinement les unités de calcul du CPU.
- **Optimisation Mémoire :** Implémenter un *Memory Pool* ou une arène d'allocation pour supprimer le surcoût de `malloc/free` identifié lors du profilage.
- **Parallélisme de Modèles :** Entraîner plusieurs instances du réseau en parallèle (par exemple pour l'exploration d'hyperparamètres), en complément du parallélisme de données déjà exploité par les mini-batches.
- **Extension aux CNN :** Intégrer des couches de convolution pour traiter des jeux de données plus complexes, une évolution naturelle envisagée pour la suite du cursus (S2).
- **Parallélisme Distribué (MPI) :** Étendre l'entraînement à plusieurs nœuds de calcul en utilisant MPI pour paralléliser le traitement des batches (*Data Parallelism*).

8 Références

References

- [1] Ian Goodfellow, Yoshua Bengio, et Aaron Courville. *Deep Learning*. MIT Press, 2016. 6, 8
- [2] Yann LeCun, Léon Bottou, Yoshua Bengio, et Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 4, 7
- [3] m1-chps.github.io/glhpc/ *UVSQ, lecture5* 2025 . 5
- [4] David E. Rumelhart, Geoffrey E. Hinton, et Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. 6
- [5] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967. 19
- [6] OpenMP Architecture Review Board. *OpenMP Application Program Interface Version 5.0*. 2018. 11
- [7] Jack J. Dongarra et al. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)*, 16(1):1–17, 1990. 11

9 Glossaire

Contexte et État de l’art

Autodifférentiation	Technique permettant d’évaluer automatiquement la dérivée d’une fonction spécifiée par un programme informatique, en appliquant récursivement la règle de la chaîne (<i>chain rule</i>).
MLP	<i>Multi-Layer Perceptron</i> (perceptron multicouche). Réseau feed-forward composé d’empilements de couches linéaires et de fonctions d’activation.
MNIST	<i>Modified National Institute of Standards and Technology</i> . Jeu de données de chiffres manuscrits (28×28) utilisé comme benchmark de classification.
HPC	<i>High-Performance Computing</i> . Ensemble des techniques et architectures visant à maximiser les performances de calcul.
Backpropagation	Algorithme fondamental d’entraînement des réseaux de neurones, calculant le gradient de la fonction de perte par rapport à chaque poids du réseau en propageant l’erreur de la sortie vers l’entrée.

Architecture et Modèles (CNN)

CNN	<i>Convolutional Neural Network</i> . Architecture de réseau de neurones utilisant des couches de convolution, adaptée aux données structurées spatialement (images).
------------	---

Implémentation : Tenseurs et Réseaux

Tensor/Matrice	Dans ce projet, structure de données fondamentale encapsulant un tableau contigu de scalaires et supportant les opérations mathématiques.
RNN	<i>Recurrent Neural Network</i> . Architecture de réseau intégrant des connexions récurrentes, adaptée aux données séquentielles.

Implémentation : Optimisation

SGD *Stochastic Gradient Descent.* Méthode d’optimisation mettant à jour les paramètres en suivant le gradient estimé sur des mini-batches.

Implémentation : Calcul et Matériel

CPU *Central Processing Unit.* Processeur généraliste exécutant le programme et orchestrant le calcul.

SIMD *Single Instruction, Multiple Data.* Technique d’optimisation matérielle permettant à un processeur d’effectuer la même opération sur plusieurs points de données simultanément.

BLAS *Basic Linear Algebra Subprograms.* Une spécification standard d’API pour les bibliothèques effectuant des opérations d’algèbre linéaire numérique de bas niveau (comme le produit matriciel).

OpenMP API multi-plateforme de support du parallélisme à mémoire partagée (multi-threading) en C/C++ et Fortran.

DAG *Directed Acyclic Graph.* Structure de données utilisée pour représenter le graphe de calcul, où les nœuds sont les opérations et les arêtes les dépendances de données.

Logits Valeurs réelles en sortie du réseau avant normalisation (par exemple avant softmax), interprétées comme des scores par classe.

One-hot Encodage d’une classe par un vecteur binaire de dimension K avec un unique 1 à l’indice de la classe, et 0 ailleurs.

Softmax Fonction transformant un vecteur de scores en distribution de probabilités, fréquemment utilisée en sortie d’un classifieur multi-classes.

Epoch Une itération complète de l’algorithme d’apprentissage sur l’ensemble du jeu de données d’entraînement.

Expérimentations : Configuration

Affinité CPU Politique de placement liant l’exécution (processus/threads) à un sous-ensemble de cœurs CPU afin de réduire la variabilité et les coûts de migration.

Warm-up Exécutions préalables avant mesure visant à stabiliser caches, prédicteur de branchement, et état d’exécution.

Expérimentations : Résultats

Speedup Rapport entre un temps de référence et un temps optimisé (par exemple T_1/T_p), mesurant l’accélération obtenue.

I/O *Input/Output* (entrées/sorties). Accès aux données (lecture/écriture) pouvant devenir un goulot d’étranglement face à un noyau de calcul optimisé.

Perspectives (GPU)

CUDA	Plateforme et modèle de programmation de NVIDIA pour l'exécution de calculs sur GPU.
GPU	<i>Graphics Processing Unit</i> . Processeur massivement parallèle, particulièrement efficace pour les opérations matricielles.

Autres concepts

Overfitting	Phénomène où un modèle apprend "par cœur" les données d'entraînement (bruit inclus) et perd sa capacité à généraliser sur de nouvelles données.
--------------------	---

A Protocole de Verrouillage des Fréquences

Afin de garantir la reproductibilité des mesures, le script suivant est exécuté avant chaque campagne de test pour fixer la fréquence du CPU à 4.0 GHz sur les cœurs physiques.

Listing 5: Commandes de configuration CPU (mode performance)

```
# 1. Passer le gouverneur en mode performance
sudo cpupower frequency-set -g performance

# 2. Verrouiller la fréquence min et max à 4.0 GHz
# (Adaptez la valeur selon votre CPU)
sudo cpupower frequency-set -u 4000MHz -d 4000MHz

# 3. Vérifier la fréquence
cpupower frequency-info | grep "current_CPU_frequency"
```

Une fois les mesures terminées, l'environnement est restauré :

Listing 6: Restauration de l'environnement

```
sudo cpupower frequency-set -d 421MHz -u 5386MHz
sudo cpupower frequency-set -g powersave
```